

Bonded Agent：五天执行计划

项目名称	Bonded Agent
团队名称	Backchannel
开发周期	5 天
参赛方向	Monad Playground 开放方向；Moss 为条件性集成项
范围决定	只实现一个具有客观判断标准的 Swap 结果担保，不做通用承诺协议。

一、产品定义

一句话介绍

Moss 可以解释 Agent 预计一笔交易会产生什么结果；Bonded Agent 进一步要求 Agent 运营方为这个预期结果锁定保证金。如果实际执行结果低于承诺，合约自动向用户赔付。

目标用户

愿意让 AI Agent 准备链上交易，但不愿仅凭 Agent 的解释或模拟结果就承担全部风险的普通链上用户。

解决的问题

模拟只能说明交易在某个时刻和某种状态下可能产生什么结果，并不是保证。模拟完成后，链上状态和价格可能在用户签名前发生变化。目前即使 Agent 给出的结果没有兑现，Agent 运营方通常也不承担经济责任。

核心用户流程

1. 用户提出固定请求，例如“把 1 MON 换成 tUSDC”。
2. Agent 构造并模拟交易。
3. Agent 运营方发布一份交易计划，写明预期输出、保证输出、最大赔付、有效期、目标合约和 calldata 哈希。
4. Agent 运营方为该计划锁定 tUSDC 保证金。
5. 用户查看交易计划与保证条款，决定是否签名。
6. BondedExecutor 执行 Swap，并测量用户实际获得的 tUSDC。
7. 实际输出达到保证值时，剩余保证金退还运营方。
8. 实际输出低于保证值、但仍在保障范围内时，合约从保证金中自动补足差额。
9. Swap 调用失败时，合约退还用户输入，并支付预先约定的失败补偿。

二、项目边界

P0：必须完成

- ☐ 只支持一种交易：测试网原生 MON 换取测试代币 tUSDC。
- ☐ 只支持一个由团队控制的 Agent 运营方。
- ☐ 在 Monad Testnet 部署一个真实运行的 BondedExecutor。
- ☐ 交易计划必须包含用户、资产、金额、保证结果、最大责任、目标合约、calldata 哈希、有效期和 nonce。

- ☐ 在 Monad Testnet 真实完成保证金锁定、释放和自动赔付。
- ☐ 分别完成“正常履约”和“结果不足自动赔付”两个链上案例。
- ☐ 防止计划过期、重复执行、保证金不足和 calldata 被替换。
- ☐ 用户签名前必须看到清晰的承诺卡片。
- ☐ 结算后展示交易链接和普通语言解释。
- ☐ 完成合约测试，并进行至少 3 次可用性测试。
- ☐ 提供 README、架构图、风险说明以及真实/模拟功能披露。

P1：P0 稳定后才允许开发

- ☐ 真正接入 Moss 的 `discover -> load -> action -> simulate` 流程。
- ☐ 使用 EIP-712 离线承诺签名代替运营方链上创建计划。
- ☐ 接入真实 Monad DEX。
- ☐ 增加担保服务费。
- ☐ 支持多个 Agent 运营方及运营方声誉。
- ☐ 接入 ERC-8004 身份。
- ☐ 支持 Swap 之外的其他操作。

明确不做

- 通用自然语言承诺。
- 由 LLM 或管理员判断是否违约。
- 保险池、跨链、治理或发币。
- 在本次黑客松中与 AnnoPilot 合并。

三、技术设计

P0 合约

MockUSDC.sol

- 作为测试输出资产和 Agent 运营方的保证金资产。
- 固定小数位；仅允许测试部署者铸造。

MockDex.sol

- 接收测试 MON，并按可配置测试汇率返回 tUSDC。
- 支持最低输出；当价格超出保证金覆盖范围时安全失败。
- 可配置价格属于模拟市场组件，必须在产品和 README 中明确标注。

BondedExecutor.sol

- 创建一次性担保计划并锁定最大赔付责任。
- 将计划绑定到特定用户、目标合约、calldata 哈希、有效期和 nonce。
- 执行目标合约调用。

- 测量输出代币余额变化。
- 从保证金中支付保障范围内的差额。
- 在外部调用失败时保留退款与赔付状态，不能因为整体回滚导致无法补偿。
- 产生 `PlanOpened`、`PlanExecuted`、`ShortfallPaid`、`PlanFailed` 和 `BondReleased` 事件。

最小计划结构

```
struct Plan {
    address user;
    address operator;
    address outputToken;
    uint256 inputAmount;
    uint256 expectedOutput;
    uint256 guaranteedOutput;
    uint256 maxCompensation;
    uint256 failureCompensation;
    address target;
    bytes32 calldataHash;
    uint256 deadline;
    uint256 nonce;
}
```

必须覆盖的合约测试

- ☐ 实际输出高于保证值：不赔付，保证金释放。
- ☐ 实际输出低于保证值：精确补足差额。
- ☐ 差额等于最大赔付：使用全部允许的保证金。
- ☐ 价格超出覆盖范围：Swap 安全失败，用户输入退回。
- ☐ 目标合约或 calldata 被修改：拒绝执行。
- ☐ 计划过期：拒绝执行。
- ☐ nonce 或计划被重复使用：拒绝执行。
- ☐ 运营方保证金不足：无法创建计划。
- ☐ 非指定用户尝试执行：拒绝执行。
- ☐ 覆盖重入和代币转账失败路径。

四、完成标准

只有以下条件全部成立，项目才算完成：

- ☐ 新用户连接课程/测试钱包后，能在签名前理解保证条款。
- ☐ 正常案例完全在 Monad Testnet 上结算。
- ☐ 结果不足案例无需管理员判断即可自动赔付。
- ☐ 前端读取真实合约事件，不使用硬编码结果。
- ☐ 保存至少两套完整交易 Hash 和浏览器链接。
- ☐ 至少 3 名测试者能回答：谁提供保证金、保证了什么、为什么发生赔付。
- ☐ 产品和 README 都明确标注真实与模拟部分。
- ☐ 备用视频完整展示正常履约与自动赔付。

五、人员职责

角色	主要职责	不可转交的责任
Researcher / 产品统筹	产品边界、承诺效度、协调、测试、Pitch	违约定义和证据表述
营销 / 全能支持	招募、竞品定位、文案、运营、Demo 制作	真实用户与证据记录的诚实性
Dev 1：合约	Solidity 架构、测试、部署、事件证据	保证金和赔付逻辑正确性
Dev 2：全栈 / Agent	Agent 流程、模拟适配、前端集成、部署	用户端到端交易流程
设计	信息层级、承诺卡片、结算状态、展示视觉	用户签名前能否理解风险

六、五天个人工作清单

Day 1：冻结机制，证明自动赔付可行

Researcher / 产品统筹

- ☐ 写出一页问题定义，删除所有非 Swap 场景。
- ☐ 冻结预期输出、保证输出、差额、失败和最大责任的定义。
- ☐ 列出真实部分、模拟部分和绝不声称的内容。
- ☐ 建立共享任务板并主持 Day 1 决策会议。

营销 / 全能支持

- ☐ 整理一页竞品与质疑清单，包括“为什么不用 minOutput”。
- ☐ 联系至少 5 名潜在测试者，确定 Day 4 的 3 个测试时段。
- ☐ 起草一句话介绍、30 秒介绍和测试邀请。
- ☐ 建立截图、交易 Hash 和反馈证据清单。

Dev 1：合约

- ☐ 完成 MockUSDC、MockDex 和 BondedExecutor 最小原型。
- ☐ 本地证明结果少 10 个代币时能精确赔付 10 个代币。
- ☐ 证明外部调用失败后仍可保存退款和补偿结算。
- ☐ 向 Dev 2 提供稳定的合约接口说明。

Dev 2：全栈 / Agent

- ☐ 用最最多半天验证 Moss 能否接入目标 Monad 环境。
- ☐ 记录 Moss 可直接使用、需要适配或必须降为 P1。
- ☐ 搭建钱包连接、计划查看和执行页面。
- ☐ 定义前端使用的结构化 Agent 输出格式。

设计

- ☐ 绘制请求、计划查看、钱包签名、等待、成功、赔付和失败状态。
- ☐ 完成承诺卡片线框，包含预期输出、保证输出、保证金、有效期和风险。
- ☐ 找 1 名非开发人员测试线框并记录误解。

- ☐ 交付桌面端和移动端组件规格。

Day 1 硬门槛

- ☐ 本地合约必须能在没有 AI 或管理员裁决的情况下计算并支付差额。
- ☐ 若失败，只能继续简化合约，禁止补充 Mock validator。

Day 2：完成各自纵向模块

Researcher / 产品统筹

- ☐ 为每个 P0 状态编写验收标准，并与 Dev 1 核对合约语义。
- ☐ 完成威胁模型：用户操纵、价格变化、重放、保证金不足和恶意目标。
- ☐ 准备测试任务和理解检查问题。
- ☐ 在所有 P0 负责人确认稳定前冻结 P1。

营销 / 全能支持

- ☐ 完成网站文案，避免保险、保证盈利或生产环境安全等表述。
- ☐ 准备 Moss、Monad、保证金、minOutput 和模拟市场 FAQ。
- ☐ 确认 Day 4 测试者与时间。
- ☐ 起草三分钟 Demo 叙事。

Dev 1：合约

- ☐ 完成计划创建、保证金锁定、执行、结果测量、赔付和释放。
- ☐ 增加事件和自定义错误。
- ☐ 完成全部必需测试。
- ☐ 执行项目已有的格式化、编译和静态检查。

Dev 2：全栈 / Agent

- ☐ 使用固定结构实现“请求 -> 计划”。
- ☐ 读取链上计划并展示在前端。
- ☐ 实现钱包执行和交易回执追踪。
- ☐ 将 UI 状态连接到真实合约事件。

设计

- ☐ 交付最终承诺卡片和交易状态组件。
- ☐ 为 bond、liability、expiry 和 shortfall 提供普通语言表达。
- ☐ 设计结算前后对比。
- ☐ 确保关键风险不会被隐藏在签名按钮之后。

Day 3：部署并跑通端到端流程

Researcher / 产品统筹

- ☐ 对照 P0 清单完成内部需求审计。

- ☐ 完成 README 与架构说明初稿。
- ☐ 核查所有产品文案是否与实现一致。
- ☐ 准备用户测试观察表。

营销 / 全能支持

- ☐ 排练正常和赔付两套 Demo。
- ☐ 准备测试发布文案和简短引导。
- ☐ 收集第一批可用截图和浏览器链接。
- ☐ 整理尚未解决的评委问题。

Dev 1 : 合约

- ☐ 将 P0 合约部署到 Monad Testnet。
- ☐ 在支持的情况下验证源码，发布地址和 ABI。
- ☐ 各执行并保存一套正常与结果不足的交易。
- ☐ 向 Dev 2 交付稳定部署清单。

Dev 2 : 全栈 / Agent

- ☐ 完成 Monad Testnet 钱包与合约集成。
- ☐ 展示等待、完成、赔付、过期和拒绝状态。
- ☐ 部署可测试的前端预览版。
- ☐ 增加错误日志，但不得泄露秘密信息。

设计

- ☐ 检查桌面和移动端预览版。
- ☐ 修复溢出、钱包状态、加载跳动和交易状态问题。
- ☐ 完成 Demo 和 README 所需视觉素材。
- ☐ 确保赔付结果醒目但不过度营销。

Day 4 : 用户测试、修复与 P1 决策

Researcher / 产品统筹

- ☐ 主持至少 3 次可用性测试。
- ☐ 记录完成情况、困惑、错误风险理解和原话。
- ☐ 选择最重要的 3 项修改并在中午前分配。
- ☐ 更新限制与证据表述。

营销 / 全能支持

- ☐ 负责测试排期、钱包和测试币支持。
- ☐ 收集获得许可的反馈与截图。
- ☐ 制作不夸大人数的测试证据页。
- ☐ 完成提交文案和 Demo 旁白。

Dev 1 : 合约

- ☐ 只修复 P0 正确性问题。
- ☐ 每次修改后重跑完整测试。
- ☐ 输出最终合约地址和事件证据表。
- ☐ P0 全部稳定后最多协助一个 P1 项目。

Dev 2 : 全栈 / Agent

- ☐ 修复最关键的用户流程问题。
- ☐ 增加拒签、网络错误、计划过期和交易失败恢复。
- ☐ 仅当 Day 1 Moss 验证成功且 P0 稳定时，接入一个真实 Moss 流程。
- ☐ 准备确定性的备用 Demo 状态。

设计

- ☐ 根据实际误解修改承诺卡片。
- ☐ 完成移动端和签名附近的风险文字。
- ☐ 准备三分钟 Demo 所需页面与画面。
- ☐ 审核已部署产品的视觉一致性。

Day 5 : 冻结、验证与提交

Researcher / 产品统筹

- ☐ 冻结范围，拒绝新增功能。
- ☐ 完成 README、威胁模型、限制和真实/模拟披露。
- ☐ 主持最终排练，准备评委问答。
- ☐ 确认提交包包含全部证据。

营销 / 全能支持

- ☐ 录制并剪辑备用 Demo 视频。
- ☐ 完成项目介绍、成员贡献和公开内容。
- ☐ 制作包含链接和截图的一页证据索引。
- ☐ 进行两次计时排练并处理超时。

Dev 1 : 合约

- ☐ 冻结合约代码，重跑编译与测试。
- ☐ 核验地址、余额、权限和浏览器链接。
- ☐ 准备干净的运营方余额和两份可演示计划。
- ☐ 记录已知安全限制。

Dev 2 : 全栈 / Agent

- ☐ 部署最终前端并核验环境变量。
- ☐ 在全新浏览器环境中运行端到端清单。

- ☐ 确认备用 RPC 和 Demo 恢复流程。
- ☐ 标记或提交最终参赛版本。

设计

- ☐ 完成桌面端和移动端最终视觉检查。
- ☐ 导出截图、架构图和封面图。
- ☐ 检查文字、对比度、加载稳定性和交易状态清晰度。
- ☐ 统一视频与 Pitch 视觉。

七、最终 Demo 剧本

1. 用户要求把 1 MON 换成 tUSDC。
2. 展示模拟结果、运营方保证输出和锁定保证金。
3. 完成一次正常执行，展示保证金释放。
4. 明确展示模拟市场价格发生变化。
5. 执行第二份计划，展示差额自动赔付。
6. 在区块浏览器中打开两组交易。
7. 最后说明边界：合约只保证一个可测量结果，不保证盈利，也不证明 Agent 在所有任务中都正确。

八、停止条件

出现以下任一情况，立即停止加功能并简化：

- 赔付依赖管理员或 LLM 判断是否违约。
- 无法通过链上代币余额变化测量差额。
- 无法防止重放或 calldata 被替换。
- 前端展示的保证没有足额保证金支持。
- Moss 集成威胁正常履约和赔付两个 P0 流程。
- 团队无法准确解释哪些市场行为属于模拟。